

在 GKE 上使用 Ray 部署多主機 TPU vLLM 推論

來源：<https://codelabs.developers.google.com/next26/aiinfra-learning-pod/screen2-advanced-inferencing-part-1?hl=zh-tw>

步驟數：12

使用 Ray 和 LeaderWorkerSet，在 GKE 的多主機 TPU 上部署 Qwen 30B 模型。

目錄

1. [簡介](#)
2. [事前準備](#)
3. [建立自訂網路](#)
4. [佈建 GKE 叢集](#)
5. [啟用 GCS Rapid Cache](#)
6. [設定 Workload Identity 和儲存空間權限](#)
7. [設定模型權重](#)
8. [建立預留 TPU 節點集區](#)
9. [部署 vLLM 服務](#)
10. [測試部署作業回應](#)
11. [清除所用資源](#)
12. [恭喜](#)

步驟 1 · 約 0 分鐘

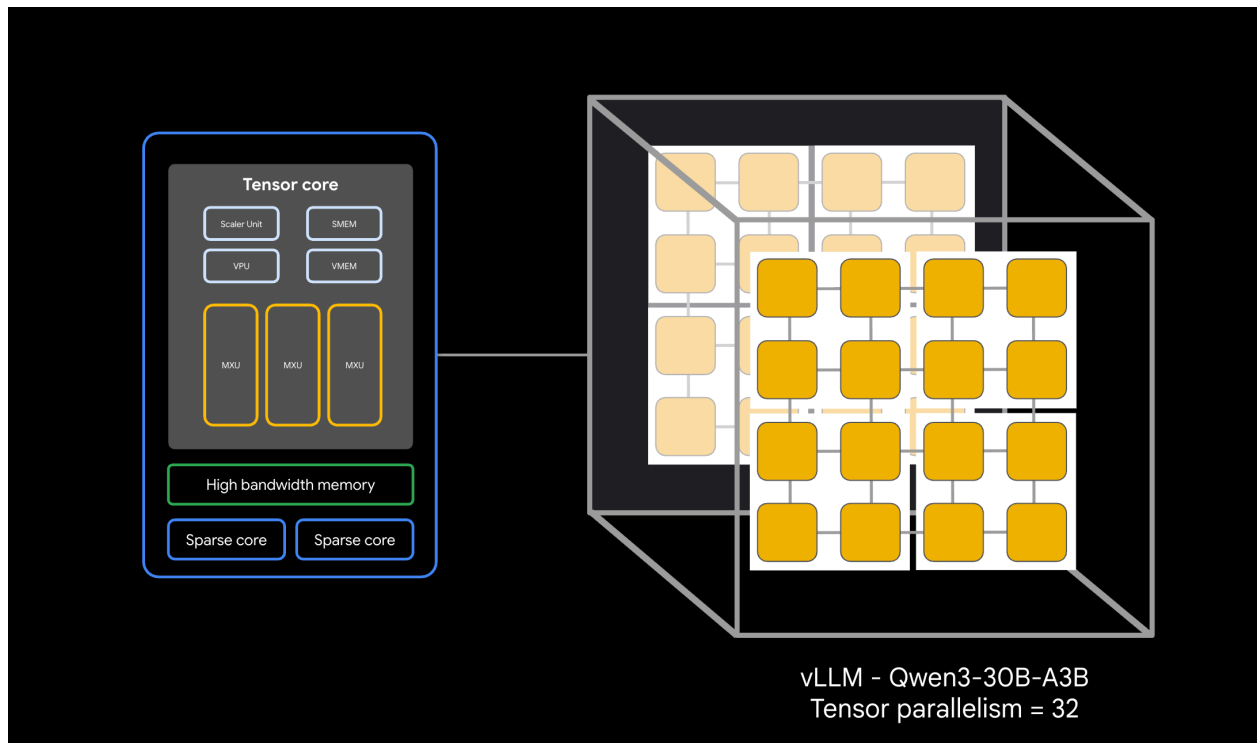
1. 簡介

在本程式碼研究室中，您將瞭解如何使用 Google Cloud TPU，在 Google Kubernetes Engine (GKE) 上部署高效能的多主機 vLLM (虛擬大型語言模型) 推論服務。您將使用 Ray 設定分散式推論，並使用 LeaderWorkerSets 在 GKE 上原生管理工作負載。

本逐步導覽會模擬正式環境設定，以便提供 [Qwen 30B](#) 等大型模型。

學習內容

- 為加速器流量建立自訂虛擬私有雲網路。
- 佈建 GKE 叢集，並使用 Ray 運算子和 GCS Fuse CSI 驅動程式。
- 初始化 GCS Rapid Cache，加快模型載入速度。
- 使用預留容量佈建多主機 TPU v6e 節點集區。
- 設定 Workload Identity，安全存取模型權重。
- 部署及測試 vLLM 引擎，提供 300 億個參數模型。



軟硬體需求

- 已啟用計費功能的 Google Cloud 專案。
- TPU v6e 資源的 **Google Cloud 預訂** (32 個晶片，`ct6e-standard-4t`)。
- 有權從來源 bucket 複製模型權重。
- Cloud Shell 或已安裝 `gcloud`、`kubectl` 和 `helm` 的本機終端機。

重要需求：本程式碼研究室需要透過特定預訂項目分配 **32 個 v6e (Trillium) 晶片**。如要取得存取權，請先與 Google Cloud 業務代表協調。

- 預計時間：60 分鐘
 - 預估費用：\$60 美元以下 (假設拆除作業迅速完成)。
-

步驟 2 · 約 5 分鐘

2. 事前準備

建立或選取 Google Cloud 專案

1. 在 [Google Cloud 控制台](#) 中，選取或建立 Google Cloud 專案。
2. 確認 Cloud 專案已啟用計費功能。

啟動 Cloud Shell

1. 點選 Google Cloud 控制台頂端的「啟用 Cloud Shell」。
2. 驗證：

□□□

```
gcloud auth list
```

3. 確認專案：

□□□

```
gcloud config get project
```

4. 視需要設定：

```
export PROJECT_ID=<YOUR_PROJECT_ID>
gcloud config set project $PROJECT_ID
```

設定環境變數

為方便執行指令，請在殼層中定義下列變數。將 `<YOUR_ZONE>` 替換為您分配到的 TPU 區域，並將 `<YOUR_RESERVATION_NAME>` 替換為預訂 ID。您必須建立 [Hugging Face 使用者存取權杖](#)，才能下載封閉式模型權重。建立完成後，請將 `<YOUR_HUGGING_FACE_TOKEN>` 替換為新建立的權杖。

```
export PROJECT_ID=$(gcloud config get-value project)
export PROJECT_NUMBER=$(gcloud projects describe ${PROJECT_ID} --
format="value(projectNumber)")
export ZONE="<YOUR_ZONE>" # e.g., us-east5-a
export REGION=${ZONE%-*}
export CLUSTER_NAME="qwen-serving-cluster"
export GVNIC_NETWORK_PREFIX="qwen-serving"
export BUCKET_NAME="inf-demo-model-storage-${PROJECT_NUMBER}"
export RESERVATION_NAME="<YOUR_RESERVATION_NAME>"
export NODE_POOL_NAME="tpu-v6e-32-resvd-pool"
export MULTIHOST_COLLECTION_NAME="tpu-6-collection"
export HF_TOKEN="<YOUR_HUGGING_FACE_TOKEN>" # Token with access to Qwen model if restricted
```

啟用 API

啟用必要的 Google Cloud 服務：

```
gcloud services enable \  
  container.googleapis.com \  
  compute.googleapis.com \  
  iam.googleapis.com \  
  cloudresourcemanager.googleapis.com
```

3. 建立自訂網路

多主機 TPU 工作負載需要特定網路設定，包括較大的 MTU 大小，才能有效率地進行加速器通訊。為叢集建立自訂虛擬私有雲網路。

1. 建立 MTU 較大的虛擬私有雲網路 (8896)：

```
gcloud compute --project=${PROJECT_ID} \
  networks create ${GVNIC_NETWORK_PREFIX}-main \
  --subnet-mode=custom \
  --mtu=8896
```

2. 建立叢集的子網路：

```
gcloud compute --project=${PROJECT_ID} \
  networks subnets create ${GVNIC_NETWORK_PREFIX}-tpu \
  --network=${GVNIC_NETWORK_PREFIX}-main \
  --region=${REGION} \
  --range=192.168.100.0/24
```

3. 建立防火牆規則，允許內部流量，讓工作人員能夠通訊：

```
gcloud compute --project=${PROJECT_ID} firewall-rules create ${GVNIC_NETWORK_PREFIX}-
allow-internal \
  --network=${GVNIC_NETWORK_PREFIX}-main \
  --allow=all \
  --source-ranges=172.16.0.0/12,192.168.0.0/16,10.0.0.0/8 \
  --description="Allow all internal traffic within the network."
```

步驟 4 · 約 8 分鐘

4. 佈建 GKE 叢集

建立標準 GKE 叢集設定，支援 GCS Fuse 掛接和 Ray Operator 工作負載。

1. 建立叢集：

```
gcloud container clusters create ${CLUSTER_NAME} \
  --project=${PROJECT_ID} \
  --location=${REGION} \
  --release-channel=rapid \
  --machine-type=e2-standard-4 \
  --network=${GVNIC_NETWORK_PREFIX}-main \
  --subnetwork=${GVNIC_NETWORK_PREFIX}-tpu \
  --num-nodes=1 \
  --gateway-api=standard \
  --enable-managed-prometheus \
  --enable-dataplane-v2 \
  --enable-dataplane-v2-metrics \
  --workload-pool=${PROJECT_ID}.svc.id.goog \
  --addons=GcsFuseCsiDriver,RayOperator \
  --enable-ip-alias
```

2. 擷取叢集憑證：

```
gcloud container clusters get-credentials ${CLUSTER_NAME} --region=${REGION}
```

3. 建立 Hugging Face Secret：安全地儲存權杖，以便下載容器存取權：

```
kubectl create secret generic hf-secret \
  --from-literal=hf_api_token=${HF_TOKEN} \
  --dry-run=client -o yaml | kubectl apply -f -
```

4. 透過 Helm 安裝 **LeaderWorkerSet (LWS)**。LWS 會管理必須一併排程的 Pod 群組：

```
helm install lws oci://registry.k8s.io/lws/charts/lws \
  --version=0.7.0 \
  --namespace lws-system \
  --create-namespace \
  --wait
```

步驟 5 · 約 3 分鐘

5. 啟用 GCS Rapid Cache

如要在服務期間加快從 Cloud Storage 讀取數十 GB 權重的速度，請建立 GCS bucket，並在可用區中啟用 GCS Rapid Cache。

1. 建立 bucket：

```
gcloud storage buckets create gs://$BUCKET_NAME \  
  --location=$REGION \  
  --uniform-bucket-level-access
```

2. 在 TPU 區域中[初始化 Rapid Cache](#)：

```
gcloud storage buckets anywhere-caches create gs://$BUCKET_NAME $ZONE \  
  --ttl=1d \  
  --admission-policy=ADMIT_ON_FIRST_MISS
```

6. 設定 Workload Identity 和儲存空間權限

設定身分連結，將權重值區安全地掛接到 GKE Pod 中，不必嵌入長期有效的金鑰。

1. 建立專屬的 IAM 服務帳戶：

```
gcloud iam service-accounts create tpu-reader-sa
```

2. 授予值區讀取權限：

```
gcloud storage buckets add-iam-policy-binding gs://${BUCKET_NAME} \
  --member="serviceAccount:tpu-reader-sa@${PROJECT_ID}.iam.gserviceaccount.com" \
  --role="roles/storage.objectAdmin"
```

3. 為 `default` 命名空間 Kubernetes 服務帳戶建立 Workload Identity 繫結：

```
gcloud iam service-accounts add-iam-policy-binding tpu-reader-
sa@${PROJECT_ID}.iam.gserviceaccount.com \
  --role="roles/iam.workloadIdentityUser" \
  --member="serviceAccount:${PROJECT_ID}.svc.id.goog[default/default]"
```

4. 為 Kubernetes SA 加入註解：

```
kubectl annotate serviceaccount default iam.gke.io/gcp-service-account=tpu-reader-
sa@${PROJECT_ID}.iam.gserviceaccount.com
```

步驟 7 · 約 8 分鐘

7. 設定模型權重

如要提供 300 億參數模型，您需要從 Hugging Face 將權重下載至 GCS 值區。如要略過 Cloud Shell 磁碟配額限制 (5 GB)，請使用 Standard Kubernetes **Job** 直接在叢集內下載，並安全地寫入已掛接的 GCS Fuse 磁碟區。

1. 部署模型下載器工作：建立並套用下列資訊清單，啟動下載作業：

```

cat <<EOF | kubectl apply -f -
apiVersion: batch/v1
kind: Job
metadata:
  name: model-downloader
spec:
  ttlSecondsAfterFinished: 60
  template:
    metadata:
      annotations:
        gke-gcsfuse/volumes: "true"
        gke-gcsfuse/memory-limit: "0"
    spec:
      serviceAccountName: default
      restartPolicy: OnFailure
      containers:
      - name: downloader
        image: python:3.10-slim
        command: ["/bin/sh", "-c"]
        args:
        - |
          pip install -U "huggingface_hub[hf_transfer]" filelock
          export HF_HUB_ENABLE_HF_TRANSFER=1

          python -c '
          import filelock

          class DummyLock:
              def __init__(self, *args, **kwargs): pass
              def __enter__(self): return self
              def __exit__(self, *args): pass
              def acquire(self, *args, **kwargs): pass
              def release(self, *args, **kwargs): pass

          filelock.FileLock = DummyLock

          from huggingface_hub import snapshot_download
          snapshot_download(
              repo_id="Qwen/Qwen3-30B-A3B",
              local_dir="/models/qwen3-30b-weights",
              local_dir_use_symlinks=False
          )
          '
        env:
        - name: HF_TOKEN
          valueFrom:
            secretKeyRef:
              name: hf-secret
              key: hf_api_token
      volumeMounts:
      - name: model-weights
        mountPath: /models

```

```
volumes:
- name: model-weights
  csi:
    driver: gcsfuse.csi.storage.gke.io
    volumeAttributes:
      bucketName: ${BUCKET_NAME}
      mountOptions: "implicit-dirs"

EOF
```

2. 監控下載作業：查看下載器 Pod 的記錄，追蹤進度：

```
kubectl logs -f job/model-downloader
```

等待工作完成，且狀態為成功。

8. 建立預留 TPU 節點集區

使用現有的容量預留項目，佈建實際的多主機 TPU 配量。

1. 執行建立指令：

```
gcloud beta container node-pools create ${NODE_POOL_NAME} \
  --project=${PROJECT_ID} \
  --cluster=${CLUSTER_NAME} \
  --region=${REGION} \
  --node-locations=${ZONE} \
  --machine-type=ct6e-standard-4t \
  --tpu-topology=4x8 \
  --num-nodes=8 \
  --scopes=https://www.googleapis.com/auth/cloud-platform \
  --reservation-affinity=specific \
  --reservation=${RESERVATION_NAME} \
  --accelerator-network-profile=auto \
  --node-labels=cloud.google.com/gke-nodepool-group-name=${MULTIHOST_COLLECTION_NAME} \
  --node-labels=cloud.google.com/gke-workload-type=HIGH_AVAILABILITY \
  --node-labels=cloud.google.com/gke-networking-dra-driver=true
```

2. 等待節點加入：您可以直接觀察節點匯總的資源調度。等待 8 個包含 `ct6e` 的節點加入 `kubectl get nodes`。

9. 部署 vLLM 服務

1. 建立網路聲明：您需要要求網路環境：

```
cat <<EOF | kubectl apply -f -
apiVersion: resource.k8s.io/v1
kind: ResourceClaimTemplate
metadata:
  name: all-netdev
spec:
  spec:
    devices:
      requests:
      - name: req-netdev
        exactly:
          deviceClassName: netdev.google.com
          allocationMode: All
EOF
```

2. 部署負載平衡器 API 端點：

```
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: Service
metadata:
  name: vllm-tpu-service
spec:
  type: LoadBalancer
  selector:
    leaderworkerset.sigs.k8s.io/name: vllm-tpu-qwen
    leaderworkerset.sigs.k8s.io/worker-index: "0"
  ports:
  - protocol: TCP
    port: 8000
    targetPort: 8000
EOF
```

3. 部署 **LeaderWorkerSet** 工作負載：這個資訊清單會跨 8 個切片主機動態啟動 Ray head/worker 彙整。

```

cat <<EOF | kubectl apply -f -
apiVersion: leaderworkerset.x-k8s.io/v1
kind: LeaderWorkerSet
metadata:
  name: vllm-tpu-qwen
spec:
  replicas: 1
  leaderWorkerTemplate:
    size: 8
    restartPolicy: RecreateGroupOnPodRestart
    workerTemplate:
      metadata:
        annotations:
          gke-gcsfuse/volumes: "true"
          gke-gcsfuse/memory-limit: "0"
        labels:
          leaderworkerset.sigs.k8s.io/name: vllm-tpu-qwen
          gke-gcsfuse/volumes: "true"
      spec:
        hostname: vllm-tpu-qwen
        serviceAccountName: default
        containers:
        - name: vllm-tpu
          image: vllm/vllm-tpu:nightly
          command: ["sh", "-c"]
          args:
            - |
              MY_TPU_IP=$(hostname -I | awk '{print $1}')
              echo "My TPU Network IP is: \${MY_TPU_IP}"

              LEADER_DNS="vllm-tpu-qwen-0.vllm-tpu-qwen"
              until getent hosts \${LEADER_DNS}; do
                echo "DNS not ready. Sleeping 5s..."
                sleep 5
              done
              LEADER_IP=$(getent hosts \${LEADER_DNS} | awk '{print $1}')

              export JAX_PLATFORMS=''
              export SCAN_TPU_CHIPS=True
              export TPU_MULTIHOST_BACKEND=ray
              export JAX_DISTRIBUTED_INITIALIZATION_TIMEOUT=300
              export LD_LIBRARY_PATH=\${LD_LIBRARY_PATH}:/usr/local/lib
              export VLLM_HOST_IP=\${MY_TPU_IP}

              if [ "\${LWS_WORKER_INDEX}" = "0" ]; then
                echo "Starting Ray Head..."
                ray start --head --port=6379 --node-ip-address=\${MY_TPU_IP} --
resources='{ "TPU": 4 }' --block &
                sleep 20
                until ray status; do sleep 5; done

                echo "Starting vLLM API Server..."

```

```

        python3 -m vllm.entrypoints.openai.api_server \
            --model=/models/qwen3-30b-weights \
            --tensor-parallel-size=32 \
            --pipeline-parallel-size=1 \
            --distributed-executor-backend=ray \
            --host=0.0.0.0 --port=8000 \
            --enforce-eager \
            --gpu-memory-utilization=0.90
    else
        ray start --address=\${LEADER_IP}:6379 --node-ip-address=\$MY_TPU_IP --
resources='{ "TPU": 4 }' --block
    fi
    ports:
    - containerPort: 8000
    - containerPort: 6379
    volumeMounts:
    - name: model-weights
      mountPath: /models
      readOnly: true
    - name: dshm
      mountPath: /dev/shm
    resources:
      claims:
      - name: net-resources
      limits:
        google.com/tpu: 4
        memory: "100Gi"
      requests:
        google.com/tpu: 4
        memory: "100Gi"
    nodeSelector:
      cloud.google.com/gke-tpu-accelerator: tpu-v6e-slice
      cloud.google.com/gke-tpu-topology: 4x8
      gke.networks.io/accelerator-network-profile: auto
    resourceClaims:
    - name: net-resources
      resourceClaimTemplateName: all-netdev
    volumes:
    - name: model-weights
      csi:
        driver: gcsfuse.csi.storage.gke.io
        volumeAttributes:
          bucketName: ${BUCKET_NAME}
          mountOptions: "implicit-dirs"
    - name: dshm
      emptyDir:
        medium: Memory

```

EOF

10. 測試部署作業回應

LeaderWorkerSet 中的所有 Pod 可能需要 5 到 10 分鐘才能提取容器映像檔、初始化 Ray，並完全 Ready。您可以監控 Pod 初始化作業，追蹤狀態：

```
kubectl get pods -l leaderworkerset.sigs.k8s.io/name=vllm-tpu-qwen -w
```

請等待所有 8 個 vllm-tpu-qwen- Pod 顯示 STATUS 為 Running，以及 READY 為 2/2，並確認負載平衡器已收到外部 IP，再繼續操作。這項作業應會在 7 到 10 分鐘內完成。

1. 擷取外部 IP：

```
export EXTERNAL_IP=$(kubectl get svc vllm-tpu-service -o  
jsonpath='{.status.loadBalancer.ingress[0].ip}')  
echo $EXTERNAL_IP
```

注意：在正式版服務中，這個端點應使用 [Identity-Aware Proxy \(IAP\)](#) 等服務加以保護

2. 使用 curl 提交推論要求：

```
curl -N -s http://$EXTERNAL_IP:8000/v1/chat/completions \  
-H "Content-Type: application/json" \  
-d '{  
  "model": "/models/qwen3-30b-weights",  
  "messages": [{"role": "user", "content": "Write a haiku about high-  
performance computing on TPUs."}],  
  "temperature": 0.7,  
  "max_tokens": 100,  
  "stream": true  
}' | sed 's/^data: //' | grep -v '\[DONE\]' | grep -v '^$' | jq -rj  
'choices[0].delta.content // empty' ; echo ""
```

您應該會看到類似 JSON 回應的輸出內容，其中包含代表所產生推論的文字！

11. 清除所用資源

如要避免系統持續向您的 Google Cloud 帳戶收費，請刪除本程式碼研究室建立的資源。

1. 刪除節點集區：

```
gcloud container node-pools delete "${NODE_POOL_NAME}" \
  --cluster="${CLUSTER_NAME}" \
  --region="${REGION}" \
  --project="${PROJECT_ID}" --quiet
```

2. 刪除叢集：

```
gcloud container clusters delete "${CLUSTER_NAME}" \
  --region="${REGION}" \
  --project="${PROJECT_ID}" --quiet
```

3. 刪除網路和防火牆設定：

```
gcloud compute firewall-rules delete \
  "${GVNIC_NETWORK_PREFIX}-allow-internal" \
  --project="${PROJECT_ID}" --quiet

gcloud compute networks subnets delete "${GVNIC_NETWORK_PREFIX}-tpu" \
  --region="${REGION}" --quiet

gcloud compute networks delete "${GVNIC_NETWORK_PREFIX}-main" --quiet
```

4. 取消繫結並刪除服務帳戶：

```

# 1. Create the cleanup script
cat << 'EOF' > clean_up_sa.sh
#!/bin/bash

# Validate that PROJECT_ID is available
if [ -z "$PROJECT_ID" ]; then
    echo "Error: PROJECT_ID environment variable is not set."
    exit 1
fi

SA_EMAIL="tpu-reader-sa@${PROJECT_ID}.iam.gserviceaccount.com"
SA_MEMBER="serviceAccount:${SA_EMAIL}"

echo "Gathering IAM policy for ${SA_EMAIL}..."

# Fetch roles assigned to this specific SA
ROLES=$(gcloud projects get-iam-policy ${PROJECT_ID} \
    --flatten="bindings[].members" \
    --filter="bindings.members:${SA_MEMBER}" \
    --format="value(bindings.role)")

if [ -z "$ROLES" ]; then
    echo "No IAM bindings found for this service account."
else
    for ROLE in $ROLES; do
        echo "Removing binding for: ${ROLE}..."
        gcloud projects remove-iam-policy-binding ${PROJECT_ID} \
            --member="${SA_MEMBER}" \
            --role="${ROLE}" --quiet > /dev/null
    done
    echo "Successfully unbound all roles."
fi

# 2. Delete the service account itself
echo "Deleting service account..."
gcloud iam service-accounts delete ${SA_EMAIL} --project=${PROJECT_ID} --quiet

echo "Cleanup complete."
EOF

# 2. Make the script executable and run it
chmod +x clean_up_sa.sh
./clean_up_sa.sh

```

5. 刪除 **GCS Bucket**：前往 Cloud 控制台，依序選取「Cloud Storage」->「Buckets」，選取「inf-demo-model-storage」，然後選擇「Delete」。

步驟 12 · 約 0 分鐘

12. 恭喜

恭喜！您已成功部署多主機 TPU 高推論率 vLLM 堆疊，並透過 Google Kubernetes Engine 原生使用 Ray。

目前所學內容

- 提供專為高速 TPU 流量量身打造的自訂路徑。
 - 使用 GCS Fuse 和區域快速快取掛接權重。
 - 透過 LeaderWorkerSets 自動調度管理多主機工作負載切片，並以原生方式同步處理。
 - 如要瞭解詳情，請參閱 [vLLM 使用者指南](#)和 [llm-d 部署指南](#)。
-